ChatGPT

Textile Manufacturing Mini-ERP for the Odoo × NMIT 24-Hour Hackathon

Executive summary

The biggest way your team loses this hackathon is by trying to build the entire textile ERP described in the problem statement. The specification touches raw-material receiving, inventory, machine scheduling, spinning/weaving/dyeing, quality, buyer updates, dispatch, worker attendance, wages, costing and month-end finance. Building all of those properly in 24 hours with four people is unrealistic. The supplied challenge itself spans nearly an entire manufacturing ERP. filecite turn0file1

Your winning scope should instead be one **deep, transactionally correct vertical slice**:

> **Buyer Order → Material Availability → Inventory Reservation → Production Routing → Roll Creation → Quality Pass/Fail → Finished Goods → Dispatch → Traceability**

Then surround that vertical slice with a lightweight dashboard, stock receipt/replenishment warning, machine status and order-cost summary.

This is unusually well aligned with what Odoo is explicitly asking for at the NMIT final. The official Odoo × NMIT page says solutions must use dynamic rather than static data, provide a responsive clean UI, robustly validate input, use intuitive navigation, and demonstrate proper team-wide Git usage. It also specifically calls backend APIs/data modelling/local databases desirable, tells teams to understand AI-generated code rather than blindly paste it, and recommends an offline/local fallback instead of depending entirely on the internet. [1] Odoo also describes these hackathons as hiring assessments where they observe what candidates can actually build and how teams brainstorm and solve problems together; it explicitly says hiring looks at talent, mindset and determination, not just project completion. [2]

The **single strongest demo moment** should not be an AI feature. It should be this:

1. Buyer places an order for **1,000 m Navy Cotton Twill**, due October 18.
2. ERP calculates that the order requires, for example, **350 kg of yarn** from your seed recipe/BOM.
3. Current usable yarn = 420 kg, so the system atomically reserves 350 kg.
4. Manufacturing order is generated.
5. Fabric moves through `WEAVING → DYEING → FINISHING`.
6. Production creates rolls `R-101`, `R-102`, `R-103`.
7. QC passes two rolls and fails one because of a colour mismatch.
8. The team deliberately tries to dispatch the failed roll.
9. **Backend refuses the dispatch with `ROLL_NOT_QC_APPROVED`.**
10. Failed roll is sent to rework; after reinspection it passes.

11. Dispatch completes, raw and finished inventory update, and the buyer order becomes `DISPATCHED`.
12. Clicking any roll displays **buyer → order → production batch → machine → processing history → quality inspection → dispatch**.

That demonstrates a real ERP rather than CRUD pages. Odoo itself models manufacturing around bills of materials, operations, manufacturing orders, work orders and work centres; it tracks work-centre capacity/productivity and supports lot-based manufacturing traceability and quality control within manufacturing operations. [3]

There is also an important domain issue hiding inside your problem statement that most competitors may miss. The prompt says Everest Textile Mills **"buys yarn"** but also says that yarn then moves through **"spinning, weaving and dyeing."** 🎛filecite🎛turn0file1🎛 Technically, spinning ordinarily converts fibre into yarn; weaving/knitting then converts yarn into fabric, followed by wet processing such as dyeing and then finishing. Cotton Incorporated's textile-learning materials similarly distinguish yarn formation/spinning from weaving/knitting, dyeing and finishing. [4]

Do not hard-code `spinning → weaving → dyeing` **for every product.** Model production as configurable process routes. For the demonstration product that starts from purchased yarn, use:

`Yarn Receipt → Weaving → Dyeing → Finishing → QC → Packing`

Keep `SPINNING` as an available operation for products starting from fibre. That single design decision makes your system look considerably more mature than a literal but incorrect interpretation of the prompt.

My recommendation for the 24-hour prototype is therefore:

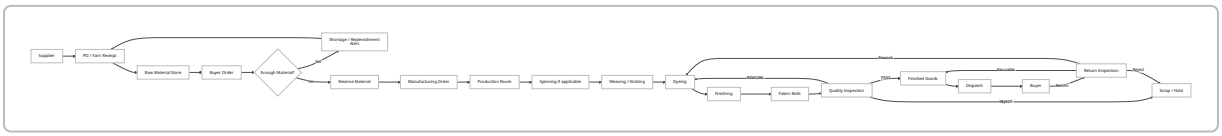
Area	Decision
Architecture	Modular monolith, not microservices
Database	PostgreSQL if already comfortable; SQLite local fallback
Manufacturing model	Configurable routing + work orders
Inventory	On-hand + reserved balance, backed by stock-movement ledger
Traceability	Material lot → manufacturing order → fabric roll → QC → dispatch
Quality	Pass / Rework / Reject with defect details
Dispatch	Only QC-approved, undispatched rolls
Accounting	Calculated order-cost summary only
Payroll	Cut unless everything else is complete
AI	Cut unless it solves a visible business problem
Demo emphasis	Business rules, traceability, failure handling and clean UX

The broader textile domain supports this direction: GOTS describes textile processing as a chain involving first processing, spinning, weaving/knitting, wet processing and manufacturing, while GS1 traceability guidance distinguishes product, batch/lot and individual-instance identification and emphasizes maintaining links when materials are transformed or repacked. [5]

Textile workflow and hackathon scope

The challenge is essentially asking you to replace notebooks, paper slips and phone calls with a shared operational state. It specifically identifies raw-material stock, machine/order status, quality records, buyer updates and worker attendance as fragmented manual processes, then asks for yarn receiving, real-time production tracking, defect inspection, buyer status visibility, dispatch control and owner-level cost visibility. 🈀filecite🈀turn0file1🈁

A realistic but hackathon-sized manufacturing flow is:



The route should be data-driven rather than embedded in frontend logic. Odoo follows the same general abstraction: a BoM defines components, manufacturing operations can be associated with work centres, and the resulting manufacturing order populates components and work orders from that configuration. [6]

Procurement and raw-material receiving. For your prototype, you do not need a complete purchasing/accounting module. A `Goods Receipt` screen that records supplier, material, lot, quantity, unit cost and received date is enough. Real ERP systems typically connect purchase orders, receipts and vendor billing; Odoo can associate received products and purchase-order lines with vendor bills. [7] A sophisticated receiving flow can place incoming goods through input and quality areas before stock becomes available, as Odoo's three-step receipt process does, but that is unnecessary for the core hackathon slice. [8]

Inventory and reservation. Do not merely store `yarn.quantity = 420`. Separate `on_hand` from `reserved`. Odoo's reservation model similarly allocates stock to orders rather than treating all on-hand stock as free, and its "At Confirmation" method only reserves when stock exists. Its forecast model conceptually computes availability from on-hand, incoming and outgoing quantities. [9] Reordering rules can then warn or trigger replenishment when forecasted inventory falls below a defined minimum. [10]

Production. Use work orders such as `WEAVING`, `DYEING`, `FINISHING`, each optionally assigned to a machine/work centre and carrying start/end times, produced quantity and state. Odoo work centres explicitly support capacity, operating costs, scheduling and efficiency tracking. [11]

Quality. Quality must be a real state transition, not a checkbox stored only on the frontend. Odoo models quality checks against inventory/manufacturing operations and supports pass/fail, measurements, photographs and other check types; quality-control points can automatically trigger checks for particular manufacturing operations. [12] Textile-specific standards reinforce the usefulness of structured defect

categories: ISO 8498 defines woven-fabric defect vocabulary, ISO 8499 does the equivalent for knitted fabrics, and ASTM D5430 covers visual inspection and grading of fabrics for commercial acceptance. [13]

For the hackathon, do **not** claim to be ASTM-compliant unless you actually implement the prescribed inspection method. Instead use a simplified defect registry:

`HOLE`, `COLOUR_MISMATCH`, `UNEVEN_WEAVE`, `STAIN`, `EDGE_DAMAGE`, `OTHER`

and optionally show a `defect_points` field as future extensibility.

Finished-roll traceability is non-negotiable. The prompt explicitly requires each roll to be traceable to its order and machine. filecitefileturn0file1 Lot/batch traceability is a standard manufacturing concept: GS1's Global Traceability Standard explains that batch/lot identification enables a group of production instances to be tracked through the supply chain, while instance-level identification can uniquely identify an individual object. [14] Odoo similarly uses lots or serial numbers to track manufactured items throughout their lifecycle. [15]

For fabric, **one generated roll ID per physical roll** is much more impressive than only tracking aggregate metres.

A roll could look like:

```
ROLL-20261010-0042

Buyer:           Orion Apparel
Order:           SO-1042
Fabric:          Cotton Twill
Colour:          Navy
Meters:          318 m

Production:
  Weaving        Loom L-04
  Dyeing         Jet Dye J-02
  Finishing      Finish Line F-01

QC:
  Result          PASS
  Inspector       QC User
  Checked         18:42

Dispatch:
  DIS-018
  Vehicle         KA-01-AB-1234
```

For returns, mimic a reverse stock movement rather than deleting the dispatch. Odoo handles customer returns as reverse transfers from a completed delivery and maintains subsequent repair/redelivery flows separately. [16]

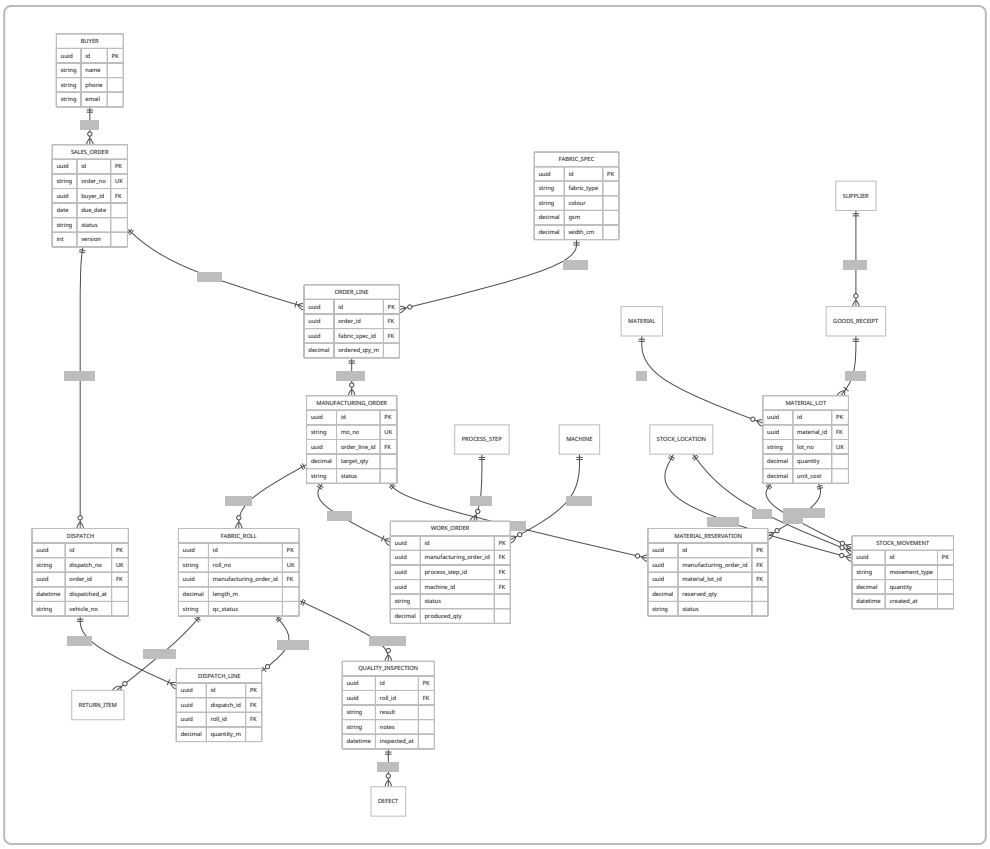
The scope cut should therefore be brutal:

Build deeply	Implement lightly	Do not build initially
Buyer orders	Supplier registry	Full general ledger
Raw inventory	Yarn receipt	GST/tax engine
Reservation	Low-stock alerts	Full payroll
Manufacturing order	Machine utilisation	HR management
Process stages	Approximate costing	Supplier portal
Roll traceability	Buyer-status view	AI chatbot
Quality/rework	Returns	Predictive maintenance
Dispatch	Simple analytics	Microservices
Order-status timeline	Basic role permissions	Blockchain
Audit trail	Cost breakdown	IoT integration

Odoo specifically warns hackathon participants to use trendy technologies only where they add real value. [1]

Core data model and business invariants

The right data model is where you separate yourself from teams building disconnected screens.



Odoo's own model justifies the abstractions rather than the exact schema: BoMs contain components and operations, work centres execute work orders, lots/serials provide traceability, and quality checks attach to manufacturing/inventory operations. 3

The **business invariants** matter more than the field list:

Invariant	Enforcement
Order quantity must be `> 0`	DB `CHECK` + request validation
Roll length must be `> 0`	DB `CHECK`
Reservation must never exceed available stock	Atomic transaction
`available = on_hand - reserved` must never be negative	Transaction/domain service
One material reservation belongs to an actual lot and MO	Foreign keys
Manufacturing cannot consume unreserved material	Domain rule
A work order cannot skip required predecessors	State-transition validator

Invariant	Enforcement
Only one active machine job per machine unless configured capacity permits otherwise	Transaction/unique constraint
Every roll belongs to exactly one manufacturing order	Non-null FK
Every produced roll inherits order traceability through the MO	FK chain
`QC_FAIL` roll cannot enter available finished stock	Domain rule
`QC_FAIL` roll cannot be dispatched	Dispatch transaction
One physical roll cannot be dispatched twice	Unique constraint on active `dispatch_line.roll_id`
Dispatched quantity cannot exceed QC-approved available quantity	Transaction
Cancelled orders cannot create new production	State rule
Status cannot be arbitrarily edited	Transition function
Same idempotency key cannot create duplicate dispatches	Unique constraint
Concurrent updates cannot silently overwrite one another	`version` optimistic lock

PostgreSQL supports primary-key, unique, foreign-key, not-null and check constraints precisely for this sort of integrity enforcement, rather than leaving every rule solely to application code. [17] SQLite also supports foreign keys, but applications should explicitly enable them because enforcement is not guaranteed merely by writing `REFERENCES`; the SQLite documentation recommends setting `PRAGMA foreign_keys = ON` rather than relying on defaults. [18]

Your state machines should be explicit:

```
ORDER
DRAFT
  ↓ confirm
CONFIRMED
  ↓ reserve
MATERIAL_RESERVED
  ↓ release
IN_PRODUCTION
  ↓ all production done
AWAITING_QC
  ↓ passed quantity available
READY_TO_DISPATCH
```

```
  ↓ dispatch
PARTIALLY_DISPATCHED
  ↓ complete
DISPATCHED
```

and:

```
ROLL
PRODUCED
  ↓
QC_PENDING
  ├── PASS ─────→ AVAILABLE
  ├── REWORK ───→ REWORKING → QC_PENDING
  └── REJECT ───→ REJECTED / SCRAPPED
```

Odoo explicitly supports failing quality checks and handling failed material through failure locations, while manufacturing scrap removes unusable material from physical inventory and records it in a separate virtual scrap location. [19]

Do **not** store only an order-level `quality_status`. Quality belongs to the physical roll. A single manufacturing order can legitimately produce some accepted and some failed output, which is exactly what your problem statement describes. filecite turn0file1

Functional priorities and API contracts

For this competition, "MUST" does not mean "everything mentioned in the PDF." It means what must work end-to-end to convincingly solve the problem and satisfy the organizer's requirements for dynamic data, validation, UI quality and proper architecture. [1]

Priority	Requirement	Judge-facing acceptance criterion
MUST	Buyer/order creation	New order persists in DB and immediately appears on dashboard
MUST	Fabric specification	Type, colour, quantity and due date validated
MUST	Raw-material inventory	Stock changes dynamically through receipt/reservation/ consumption
MUST	Material availability	System visibly calculates available vs required
MUST	Reservation	Two orders cannot reserve the same unavailable stock
MUST	Manufacturing order	Confirmed order can generate production
MUST	Configurable stages	Product route visible as ordered operations
MUST	Machine assignment	Current order/machine relationship displayed

Priority	Requirement	Judge-facing acceptance criterion
MUST	Work-order progress	Start/complete updates production state
MUST	Roll generation	Physical roll IDs generated and traceable
MUST	Quality	Each roll passes, reworks or rejects independently
MUST	Dispatch gate	Non-passed roll is rejected server-side
MUST	Dispatch	Shipped rolls removed from available finished goods
MUST	Order status	Buyer order shows current location/status
MUST	Traceability	Roll traces back to order, MO and machines
MUST	Input validation	Invalid qty/state transition gets useful error
SHOULD	Low-stock alert	Reservation can trigger visual warning
SHOULD	Goods receipt	New yarn lot enters stock
SHOULD	Cost estimate	Material + machine + labour approximation
SHOULD	Rework loop	Failed roll can return to production/QC
SHOULD	Buyer status page	Read-only order progress page
SHOULD	Machine dashboard	Busy/idle/down state visible
SHOULD	Return	Reverse-dispatch movement
SHOULD	Audit trail	Shows actor/time/status changes
OPTIONAL	Worker attendance	Basic shift records
OPTIONAL	Wages	Derived labour summary
OPTIONAL	Photo defect evidence	Image on QC record
OPTIONAL	QR roll labels	Opens traceability screen
OPTIONAL	Notifications	Simulated buyer status notification
OPTIONAL	AI	Only if core system is finished

The allocation/reservation behavior is not invented for the demo: Odoo likewise distinguishes inventory reservation and supports replenishment based on forecasted inventory thresholds. [20]

A pragmatic API surface could be:

```
POST   /api/orders
GET    /api/orders/{orderId}
POST   /api/orders/{orderId}/confirm
POST   /api/orders/{orderId}/reserve-materials
```

```
POST   /api/orders/{orderId}/release-production

GET    /api/inventory
POST   /api/inventory/receipts

GET    /api/manufacturing-orders/{moId}
POST   /api/work-orders/{workOrderId}/start
POST   /api/work-orders/{workOrderId}/complete

POST   /api/rolls
GET    /api/rolls/{rollId}/trace
POST   /api/rolls/{rollId}/quality-check

POST   /api/dispatches
GET    /api/dispatches/{dispatchId}

POST   /api/returns
```

Create buyer order

```
POST /api/orders
Idempotency-Key: create-order-client-42
```

```
{
  "buyer_id": "buyer-orion",
  "due_date": "2026-10-18",
  "items": [
    {
      "fabric_spec_id": "cotton-twill-navy-240gsm",
      "quantity_m": 1000
    }
  ]
}
```

Response:

```
{
  "id": "so_1042",
  "order_no": "SO-1042",
  "status": "DRAFT",
  "buyer": {
    "id": "buyer-orion",
    "name": "Orion Apparel"
  },
```

```
  "items": [
    {
      "fabric_spec_id": "cotton-twill-navy-240gsm",
      "quantity_m": 1000
    }
  ],
  "version": 1
}
```

Reserve raw material

```
POST /api/orders/so_1042/reserve-materials
Idempotency-Key: reserve-so-1042-v1
```

```
{
  "strategy": "FIFO"
}
```

Success:

```
{
  "order_id": "so_1042",
  "status": "MATERIAL_RESERVED",
  "requirements": [
    {
      "material": "Cotton Yarn 30s Navy",
      "required_kg": 350,
      "reserved_kg": 350,
      "allocations": [
        {
          "lot_no": "YARN-20261009-A",
          "quantity_kg": 300
        },
        {
          "lot_no": "YARN-20261010-B",
          "quantity_kg": 50
        }
      ]
    }
  ]
}
```

Failure:

```
409 Conflict
```

```
{
  "error": {
    "code": "INSUFFICIENT_MATERIAL",
    "message": "Not enough Cotton Yarn 30s Navy to reserve this order.",
    "details": {
      "required_kg": 350,
      "available_kg": 210,
      "shortage_kg": 140
    }
  }
}
```

Release production

```
POST /api/orders/so_1042/release-production
```

Response:

```
{
  "manufacturing_order": {
    "id": "mo_402",
    "mo_no": "MO-0402",
    "status": "READY",
    "target_quantity_m": 1000,
    "route": [
      {
        "sequence": 10,
        "operation": "WEAVING",
        "work_center": "Loom Section"
      },
      {
        "sequence": 20,
        "operation": "DYEING",
        "work_center": "Dye House"
      },
      {
        "sequence": 30,
        "operation": "FINISHING",
        "work_center": "Finishing Line"
      }
    ]
```

```
  }
}
```

This mirrors the BoM/work-order abstraction used in Odoo manufacturing, where components and operations populate manufacturing orders. [6]

Complete work order

```
POST /api/work-orders/wo_990/complete
```

```
{
  "expected_version": 3,
  "produced_quantity_m": 1005,
  "waste_quantity_m": 5,
  "operator_note": "Completed without machine stoppage"
}
```

Response:

```
{
  "id": "wo_990",
  "operation": "WEAVING",
  "machine": "LOOM-04",
  "status": "COMPLETED",
  "produced_quantity_m": 1005,
  "waste_quantity_m": 5,
  "version": 4
}
```

Version mismatch:

```
409 Conflict
```

```
{
  "error": {
    "code": "VERSION_CONFLICT",
    "message": "Work order was modified by another user.",
    "current_version": 4
  }
}
```

QC a roll

```
POST /api/rolls/roll_103/quality-check
```

```
{
  "result": "FAIL",
  "disposition": "REWORK",
  "defects": [
    {
      "type": "COLOUR_MISMATCH",
      "severity": "MAJOR",
      "notes": "Shade outside buyer reference"
    }
  ]
}
```

Response:

```
{
  "roll_id": "roll_103",
  "roll_no": "ROLL-0103",
  "qc_status": "REWORK_REQUIRED",
  "inventory_location": "QC_HOLD"
}
```

Odoo supports quality checks against manufactured items and supports both pass/fail checks and structured measurements or pictures when required. [21]

Dispatch approved rolls

```
POST /api/dispatches
Idempotency-Key: dispatch-so-1042-01
```

```
{
  "order_id": "so_1042",
  "roll_ids": [
    "roll_101",
    "roll_102"
  ],
  "vehicle_no": "KA01AB1234",
```

```
  "driver_name": "Ramesh"
}
```

Success:

```
{
  "dispatch_id": "disp_18",
  "dispatch_no": "DIS-0018",
  "order_id": "so_1042",
  "rolls": [
    "ROLL-0101",
    "ROLL-0102"
  ],
  "quantity_m": 682,
  "order_status": "PARTIALLY_DISPATCHED"
}
```

Trying to include failed `roll_103`:

```
409 Conflict
```

```
{
  "error": {
    "code": "ROLL_NOT_QC_APPROVED",
    "message": "ROLL-0103 cannot be dispatched.",
    "details": {
      "qc_status": "REWORK_REQUIRED"
    }
  }
}
```

That error is worth demonstrating live.

Use one common error envelope everywhere:

```
{
  "error": {
    "code": "MACHINE_READABLE_CODE",
    "message": "Human readable explanation",
    "field_errors": {},
    "details": {}
  },
```

```
  "request_id": "req_01HT..."
}
```

Recommended HTTP mapping:

Situation	Status
Invalid type/range	422
Authentication missing	401
Permission denied	403
Record not found	404
Illegal state transition	409
Insufficient stock	409
Version conflict	409
Duplicate request with same idempotency key	Return original successful response
Unexpected backend failure	500

HTTP defines `PUT`, `DELETE` and safe methods as idempotent; for creation commands such as dispatch `POST`, using an application-level idempotency key is a practical way to make retries safe. [22]

Transaction integrity, database, security and performance

This is where many otherwise good hackathon projects break. Two browser tabs reserve the same stock. The UI sends the request twice. Someone dispatches the same roll twice. A stale production page overwrites another member's update.

Those are not UI problems. They are database consistency problems.

Database decision

Option	Strength	Weakness	Hackathon verdict
SQLite	Almost zero setup; one file; excellent offline fallback	Single-writer concurrency model; locking differs from production DBs	**Excellent fallback / acceptable primary for low concurrency**
PostgreSQL	Real row-level locks, constraints and stronger concurrent-write behavior	More setup and another service to keep alive	**Best primary if team already knows it**

Option	Strength	Weakness	Hackathon verdict
In-memory / static JSON	Fastest initially	No persistence/integrity; directly conflicts with organizer expectation	**Do not use**

The NMIT hackathon explicitly tells participants to use dynamic data rather than static JSON and to plan local/offline solutions. [1]

My recommended deployment layout is:

```
Frontend
   │
   ▼
Backend API / Domain Services
   │
   ├──── PostgreSQL       primary
   │
   └──── SQLite           local emergency profile
```

Keep the backend's persistence layer behind repositories/services so the business rules do not depend on PostgreSQL-only behaviour unless necessary.

PostgreSQL inventory reservation

Use either a conditional atomic update or explicit row locking.

A straightforward lock pattern:

```sql
BEGIN;

SELECT on_hand, reserved
FROM inventory_balance
WHERE material_id = :material_id
  AND location_id = :location_id
FOR UPDATE;

/* Application verifies:
   on_hand - reserved >= requested_quantity
*/

UPDATE inventory_balance
SET reserved = reserved + :requested_quantity,
    version = version + 1
WHERE material_id = :material_id
```

```
  AND location_id = :location_id;

INSERT INTO material_reservation (
    id,
    manufacturing_order_id,
    material_lot_id,
    reserved_quantity
)
VALUES (
    :reservation_id,
    :mo_id,
    :lot_id,
    :requested_quantity
);

COMMIT;
```

PostgreSQL's `SELECT ... FOR UPDATE` prevents conflicting writers from modifying/locking the selected row until the current transaction resolves, which is appropriate for inventory allocation. [23] PostgreSQL's default `READ COMMITTED` mode gives each command a committed snapshot and has defined behavior when concurrent writers target the same record. [24]

An even simpler portable pattern is:

```
UPDATE inventory_balance
SET reserved = reserved + :quantity,
    version = version + 1
WHERE material_id = :material_id
  AND location_id = :location_id
  AND (on_hand - reserved) >= :quantity;
```

Then require:

```
affected_rows == 1
```

Otherwise return `INSUFFICIENT_MATERIAL`.

That makes the eligibility check and update one SQL operation.

SQLite reservation

SQLite does not give you PostgreSQL-style row-level `FOR UPDATE`. Use a short explicit write transaction:

```sql
BEGIN IMMEDIATE;

UPDATE inventory_balance
SET reserved = reserved + :quantity,
    version = version + 1
WHERE material_id = :material_id
  AND location_id = :location_id
  AND (on_hand - reserved) >= :quantity;

/* Verify affected_rows == 1. */

INSERT INTO material_reservation (...);

COMMIT;
```

SQLite documents that `BEGIN IMMEDIATE` starts the write transaction immediately, preventing another connection from acquiring a concurrent writer transaction; WAL mode allows readers and a writer to coexist better than rollback-journal mode. [25]

At application startup:

```sql
PRAGMA foreign_keys = ON;
PRAGMA journal_mode = WAL;
PRAGMA busy_timeout = 5000;
```

Foreign-key enforcement should be explicitly enabled in SQLite. [18]

Optimistic locking

Give mutable workflow records a `version` integer:

```sql
UPDATE work_order
SET
    status = :new_status,
    version = version + 1
WHERE id = :id
  AND version = :expected_version;
```

If no row updates:

```
{
  "error": {
    "code": "VERSION_CONFLICT"
```

```
  }
}
```

Do this for:

```
sales_order
manufacturing_order
work_order
fabric_roll
```

Do not waste time adding it to static lookup tables.

Idempotency

Create:

```
CREATE TABLE idempotency_request (
    key VARCHAR(128) PRIMARY KEY,
    operation VARCHAR(80) NOT NULL,
    response_status INTEGER,
    response_body TEXT,
    created_at TIMESTAMP NOT NULL
);
```

Inside the same command transaction:

```
1. Insert idempotency key.
2. If it already exists → return stored result.
3. Perform command.
4. Save response.
5. Commit.
```

SQLite's UPSERT functionality and PostgreSQL's `ON CONFLICT` semantics are useful for this pattern. SQLite explicitly models its UPSERT syntax after PostgreSQL. [26]

Database-level constraints

Implement at least:

```
CHECK (ordered_quantity > 0)
CHECK (roll_length > 0)
CHECK (reserved >= 0)
```

```
CHECK (on_hand >= 0)

UNIQUE (order_no)
UNIQUE (mo_no)
UNIQUE (roll_no)
UNIQUE (dispatch_no)

FOREIGN KEY (...)
```

Primary, unique, check and foreign-key constraints are deliberately designed to maintain these relational invariants. [27]

Dispatch transaction

Dispatch is the most important transaction after reservation:

```
BEGIN

lock/check every requested roll

for each roll:
    require qc_status == PASS
    require dispatch_status == NOT_DISPATCHED
    require order_id == requested order

create dispatch
create dispatch_lines
move stock FG → DISPATCHED
mark rolls dispatched
recalculate order fulfilment

COMMIT
```

Nothing should happen halfway.

Security that is worth implementing in a hackathon

Do not build enterprise IAM. Do enforce server-side rules.

Use four operational roles if authentication already exists:

```
SALES
PRODUCTION
QUALITY
```

```
DISPATCH
ADMIN
```

A quality user should not mutate stock reservations. A dispatch user should not force a failed roll to `PASS`. Authorization must be enforced at the protected backend operation, not merely by hiding frontend buttons; OWASP explicitly recommends checking permission at the protected resource and denying by default. [28]

Validate syntax **and semantics**:

```
quantity_m          > 0 and bounded
due_date            valid date
colour              bounded string / lookup
fabric_type         existing reference
work_order_status   enum
QC result           PASS | FAIL
vehicle_no          bounded text
roll IDs            actual rolls belonging to order
```

OWASP recommends server-side validation of expected type, range, format and semantic relationships rather than trusting client validation. [29] This is also an explicit Odoo × NMIT requirement. [1]

Never concatenate user input into SQL. Use your ORM or parameterized queries; OWASP identifies parameterized queries as the primary SQL-injection defence. [30]

If you implement passwords, do not store plaintext or simple SHA hashes. OWASP recommends slow password hashing such as Argon2id, bcrypt or PBKDF2. [31]

For hackathon performance, set **engineering targets**, not elaborate infrastructure:

```
Order list:             < 300 ms locally
Order detail:           < 300 ms locally
State-changing API:     < 500 ms locally
Dashboard initial load: < 1 s locally
```

Those are targets for your team, not textile-industry standards.

Index the fields your demo actually queries:

```
CREATE INDEX idx_order_buyer
    ON sales_order(buyer_id);

CREATE INDEX idx_order_status
    ON sales_order(status);
```

```sql
CREATE INDEX idx_work_order_mo
    ON work_order(manufacturing_order_id);

CREATE INDEX idx_work_order_machine_status
    ON work_order(machine_id, status);

CREATE INDEX idx_roll_mo
    ON fabric_roll(manufacturing_order_id);

CREATE INDEX idx_roll_qc_status
    ON fabric_roll(qc_status);

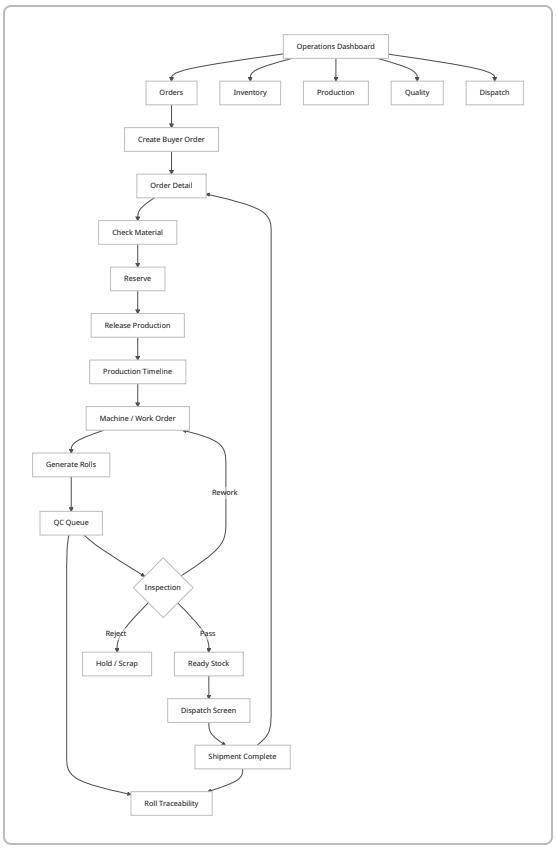
CREATE INDEX idx_reservation_lot
    ON material_reservation(material_lot_id);

CREATE INDEX idx_dispatch_order
    ON dispatch(order_id);
```

Indexing referencing foreign-key columns is often important for efficient relationship checks and lookups; PostgreSQL's documentation notes that foreign keys do not automatically create every useful index on referencing columns. [32]

UI, automated proof and demo deployment

Do not make twelve menu items. The frontend should expose the lifecycle visually.



The **dashboard** should answer in under five seconds:

```
┌──────────────────────────────────────────────────────────────┐
│ Everest Textile Mills                                        │
├──────────────┬──────────────┬──────────────┬─────────────────┤
│ Orders       │ In Production│ QC Pending   │ Ready Dispatch  │
│    12        │      5       │     7        │       3         │
├──────────────┴──────────────┴──────────────┴─────────────────┤
│ LOW STOCK                                                    │
│ Cotton Yarn Navy     70 kg available     ⚠ Reorder           │
├──────────────────────────────────────────────────────────────┤
│ ACTIVE ORDERS                                                │
│ SO-1042 Orion │ Navy Twill │ Dyeing      │ Due Oct 18        │
│ SO-1043 Nova  │ Grey Poplin│ QC          │ Due Oct 17        │
│ SO-1044 Vega  │ Blue Denim │ Ready Ship  │ Due Oct 16        │
└──────────────────────────────────────────────────────────────┘
```

The **order detail page** is your central demo screen:

```
SO-1042 · Orion Apparel
1000 m · Navy Cotton Twill · Due Oct 18
```

```
MATERIAL
Required 350 kg
Reserved 350 kg               ✓ READY

PRODUCTION
✓ Weaving   LOOM-04        1005 m
✓ Dyeing    JET-02         1002 m
● Finishing FINISH-01       682 m / 1000 m

QUALITY
ROLL-0101    340 m    PASS
ROLL-0102    342 m    PASS
ROLL-0103    318 m    REWORK - Colour mismatch

DISPATCH
682 / 1000 m ready

[ Traceability ] [ Create Dispatch ]
```

Odoo's current work-centre UI and manufacturing-order documentation are useful visual references for how production work can be presented, while the Odoo Quality documentation shows how quality checks are surfaced directly within manufacturing/inventory work. [33]

Your QC interface should require only seconds:

```
ROLL-0103
318 metres
Order SO-1042
Machine history: LOOM-04 → JET-02 → FINISH-01

Inspection Result
( ) PASS
(•) FAIL

Defect:
[ Colour mismatch ▼ ]

Disposition:
[ Rework ▼ ]

Notes:
[ Shade outside reference sample ]

[ Submit Inspection ]
```

The organizer explicitly cares about clean, consistent, responsive design and intuitive navigation, so UI polish is not cosmetic here. [1]

Automated tests are part of the demo strategy. You do not need 95% coverage. You need tests that prove your ERP cannot enter stupid states.

Test	Setup	Expected
Invalid order qty	`quantity=0`	`422`
Exact-stock reservation	Available=350, request=350	Success
Stock shortage	Available=349, request=350	`409`
Concurrent reservation	Two requests compete for same 350	Only valid allocation succeeds
Duplicate reservation retry	Same idempotency key twice	Same response; one reservation
Unreserved production	MO lacks materials	Start blocked
Stage-order violation	Start dyeing before weaving complete	`409`
Machine conflict	Start second incompatible WO	Blocked
Roll traceability	Create roll from MO	Order/MO history present
QC pass	Inspect roll PASS	FG available increases
QC fail	Inspect roll FAIL	Not FG-available
Dispatch failed roll	QC=FAIL	`409`
Dispatch same roll twice	First dispatched	Second blocked
Partial dispatch	682/1000 shipped	Order stays partial
Final dispatch	Remaining quantity shipped	`DISPATCHED`
Rework flow	FAIL → rework → PASS	Roll becomes dispatchable
Stale update	Old version number	`409 VERSION_CONFLICT`
Roll wrong order	Dispatch foreign roll	`409`
DB rollback	Error midway through dispatch	No partial stock mutation
Foreign-key violation	Unknown order/MO ID	Database rejects

Use three classes of test:

```
Unit
  state-transition rules
  quantity calculations
  BOM/material calculation
```

```
  cost calculations

Integration
  reservation transaction
  production → roll
  QC → inventory
  dispatch transaction
  idempotency

Smoke
  application starts
  database migrates
  seed loads
  login works
  order flow completes
```

The **one concurrency test** is worth more than twenty trivial controller tests.

Pseudo-test:

```
def test_two_orders_cannot_overreserve_same_stock():
    # Given only 100 kg is available.
    # Two workers concurrently request 80 kg each.
    #
    # Valid outcomes:
    #   one succeeds, one receives conflict
    #
    # Invalid outcome:
    #   both reserve 80 kg and inventory becomes -60 kg
    ...
```

For demo data, do not rely on manual entry. Create a deterministic reset command:

```
./scripts/demo-reset.sh
```

It should:

```
drop/reset demo DB
run migrations
seed:
  3 buyers
  2 suppliers
  5 materials
  5 material lots
```

```
  4 fabric specs
  6 machines
  production routes
  3 existing orders
  one low-stock material
  one order ready for the live lifecycle
```

Your application must survive internet failure. That is not paranoia: Odoo explicitly recommends local/offline solutions for the hackathon. [1]

The fallback hierarchy should be:

```
Primary:
Laptop → frontend → local backend → local PostgreSQL

Fallback:
Laptop → frontend → local backend → SQLite demo DB

Emergency:
pre-seeded stable commit + local DB snapshot
```

Do not make your judges wait while somebody troubleshoots cloud DNS.

Before presentation, have:

```
git tag: demo-final
known-good DB snapshot
demo-reset script
two browsers already open
presentation PDF exported locally
README with one-command startup
screen recording of successful flow as absolute last-resort evidence
```

The demo sequence should be rehearsed to approximately five minutes even if you later receive more presentation time.

Suggested story:

```
0:00–0:30
"Everest currently runs inventory, QC and production through notebooks,
paper slips and phone calls. Our system makes one order traceable from
buyer request to physical dispatch."

0:30–1:15
```

```
Create SO-1042.
Show live material requirement.
Reserve yarn.

1:15–2:00
Release manufacturing order.
Show configurable weaving → dyeing → finishing route.
Assign machines.
Complete stages.

2:00–3:00
Generate three rolls.
Pass two.
Fail one for colour mismatch.
Show it moved to QC hold/rework.

3:00–3:40
Attempt to dispatch all three.
System rejects failed roll.
This is the "proof" moment.

3:40–4:20
Rework/reinspect.
Dispatch approved rolls.
Inventory/order status updates.

4:20–5:00
Open roll traceability.
Show buyer → order → machines → QC → dispatch.
Finish on dashboard/cost impact.
```

The psychology is straightforward. Odoo's own explanation of the program says the hackathon is used to observe real coding/problem-solving and teamwork, and the NMIT event page explicitly prioritizes dynamic data, validation, Git, UI quality and understanding your own code. [34]

So do not spend presentation time bragging about:

> “React + FastAPI + PostgreSQL + Tailwind + Docker.”

Most competitors can say that.

Show instead:

> “A failed roll physically cannot be shipped.”

> “Two orders cannot reserve the same yarn.”

> “I can trace this buyer complaint back to the specific production batch and machines.”

That is what makes the software look like manufacturing software.

Team execution, risk management and cut strategy

Use four sharply separated owners. Do **not** let everyone work everywhere.

Member	Primary ownership	Secondary
A — System Lead	Schema, domain rules, order/reservation APIs	Integration decisions
B — Production Backend	MO, work orders, machines, rolls, QC, dispatch	Seed data
C — Frontend Lead	Dashboard, order flow, production/QC UI	Responsive design
D — Integration/QA/ DevOps	API integration, tests, demo reset, deployment	Traceability UI + pitch

Everybody commits through Git. That matters because the official NMIT requirements explicitly say that proper version control means one team member alone should not manage the repository. [1]

Use short branches:

```
feat/order-reservation
feat/production
feat/qc-dispatch
feat/dashboard
fix/dispatch-lock
```

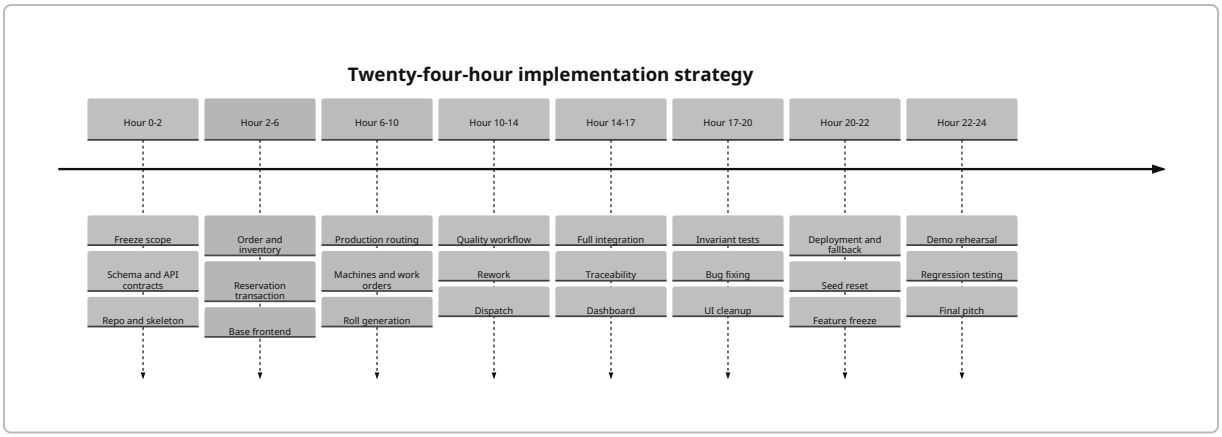
Merge continuously.

Never allow:

```
member A: "I'll build all backend and merge tonight."
member B: "I'll build all frontend separately."
```

That produces a spectacular integration disaster around hour eighteen.

The official event lists the final as a 24-hour coding round for teams of up to four, with the surprise final-round problem revealed on site. [1] Your operating timeline should be:



The hour-by-hour command plan:

Hour	Member A	Member B	Member C	Member D	Exit criterion
H0	Extract invariants	Analyse production states	Sketch UI	Set repo/ board	Scope frozen
H1	ER/schema	API contract	UI shell	Seed specification	Architecture agreed
H2	Migrations/models	Work-order models	Dashboard shell	CI/start scripts	App + DB boots
H3	Order API	Machine/route API	Order list/ create	API test harness	Order persists
H4	Inventory API	MO creation	Order detail	Seed inventory	Stock visible
H5	Reservation transaction	Production route	Material UI	Reservation tests	Atomic reserve works
H6	Shortage handling	Work-order start	Production board	Concurrency test	Shortage blocked
H7	Optimistic locking	Work-order complete	Machine cards	Integration	First production flow
H8	Review/domain cleanup	Roll creation	Production timeline	Smoke suite	**Checkpoint one**
H9	Stock movements	Roll traceability data	Roll UI	Seeds	Trace chain exists
H10	Support B	QC backend	QC queue	QC tests	Pass/fail works
H11	Integration	Rework state	QC modal	Full-flow test	Rework works

Hour	Member A	Member B	Member C	Member D	Exit criterion
H12	Dispatch transaction review	Dispatch backend	Dispatch screen	Dispatch tests	Dispatch gate works
H13	Idempotency	Partial dispatch	Order progress	Retry tests	Duplicate safe
H14	Cost summary	Scrap/hold	Dashboard KPIs	Integration	**Core MVP done**
H15	Trace query	Returns or buyer view	Traceability UI	Full smoke	Vertical slice complete
H16	Fix hardest bug	Fix hardest bug	Responsive cleanup	Regression	**No new risky features**
H17	Audit trail	Low-stock alert	Empty/error states	Test failures	Demo-worthy UI
H18	Review indexes	Performance cleanup	UX polish	Startup/ fallback	Stable branch
H19	API error cleanup	Seed consistency	Visual consistency	Reset command	Deterministic reset
H20	Feature freeze	Feature freeze	Feature freeze	Deployment	**Code freeze**
H21	All members run demo separately				Any member can operate
H22	Judge Q&A prep	Domain Q&A	Presentation	Regression	Demo rehearsed
H23	Final fixes only	Final fixes only	Final fixes only	Backup/tag	Stable tagged build
H24	Stop coding	Stop coding	Stop coding	Reset demo	Present

At **H8**, cut these first if behind:

```
notifications
photographs
returns
machine downtime analytics
cost charts
QR codes
```

At **H14**, if the complete buyer-to-dispatch flow is not working, immediately cut:

```
worker attendance
wages
procurement PO workflow
buyer portal
advanced reports
anything AI
```

At **H18**, no feature is allowed into the branch unless removing it would break the demo.

At **H20**, feature development ends.

The reason is simple: the organizer explicitly values actual functional software and says candidates should understand what they build rather than blindly adopting snippets or fashionable technology. [35]

A realistic risk matrix:

Risk	Probability	Impact	Prevention	Cut/fallback
Team over-scopes ERP	Very high	Critical	Freeze vertical slice H1	Kill accounting/payroll
Front/back integration fails	High	Critical	API contract H1; merge early	Simplify screens
Double reservation	Medium	Critical	Transaction + concurrency test	Serialize reservation
Dispatch inconsistency	Medium	Critical	One transaction	Remove partial dispatch
SQLite writer contention	Medium	Medium	Short transactions + WAL	PostgreSQL primary
Postgres setup fails	Medium	High	SQLite profile	Run local SQLite
Cloud/internet fails	Medium	Critical	Local deployment	Local-only demo
Broken demo seed	Medium	High	Deterministic reset	DB snapshot
Teammates edit same files	High	Medium	Ownership boundaries	Reassign files immediately
UI unfinished	Medium	High	UI owner starts H2	Reduce number of screens
AI-generated broken code	High	High	Review every change	Delete feature
Sleep/fatigue bugs	High	Medium	Stagger short rests	Freeze earlier

Risk	Probability	Impact	Prevention	Cut/fallback
Reviewer asks domain question	Medium	Medium	Learn invariants	Explain route configurability
Reviewer asks scalability	Medium	Medium	PostgreSQL path/ indexes	Explain prototype limits
Reviewer asks security	Medium	Medium	Server validation/RBAC	Show rejection tests

The most valuable contingency is **not more code; it is a smaller system that cannot break**.

Your "minimum winning build" is exactly:

```
LOGIN / DEMO USER
       ↓
DASHBOARD
       ↓
CREATE BUYER ORDER
       ↓
CHECK / RESERVE YARN
       ↓
GENERATE MANUFACTURING ORDER
       ↓
WEAVING → DYEING → FINISHING
       ↓
CREATE IDENTIFIED FABRIC ROLLS
       ↓
QC PASS / FAIL / REWORK
       ↓
DISPATCH ONLY PASSED ROLLS
       ↓
TRACE ROLL BACK TO ORDER + MACHINE
```

Everything outside that chain is subordinate.

Strategic verdict and primary research sources

Textile is the right problem only if you build it as a controlled operational system rather than a collection of CRUD pages. Its strength for a hackathon is that one buyer order naturally connects multiple modules—inventory, production, machines, traceability, quality and logistics—allowing you to demonstrate integration and business invariants in a single story. The supplied problem explicitly describes those connections. filecite turn0file1

The biggest competitive mistake would be implementing separate pages for "Inventory", "Machines", "Quality", "Workers" and "Finance" with no shared lifecycle. Judges can see CRUD immediately.

The architecture should instead revolve around:

Order as the business thread.

and:

Roll as the physical traceability thread.

That gives you two killer queries:

```
Where is buyer order SO-1042 right now?
```

and:

```
Where did physical roll ROLL-0103 come from?
```

A proper system answers both instantly.

The research basis for the implementation is mainly official/primary material:

Source	What to borrow
Odoo × NMIT official event requirements [1]	Dynamic data, responsive UI, validation, Git, local fallback
Odoo hackathon programme [2]	Teamwork/problem-solving/hiring context
Odoo BoM documentation [36]	Components + configurable production operations
Odoo manufacturing/work-order documentation [37]	MO → work orders
Odoo work-centre documentation [11]	Machine/work-centre capacity and performance
Odoo inventory reservation documentation [9]	Reservation semantics
Odoo replenishment documentation [38]	Low-stock/forecast logic
Odoo lot/serial traceability [15]	Manufacturing traceability
Odoo quality checks/QCPs [12]	QC attached to operations
Odoo scrap workflow [39]	Reject/scrap inventory handling
Odoo returns/repair workflow [16]	Reverse logistics
Cotton Incorporated / CottonWorks [4]	Spinning, weaving, dyeing, finishing domain structure

Source	What to borrow
GOTS processing chain [40]	Textile processing-stage context
ISO 8498 / ISO 8499 [41]	Woven/knitted defect vocabulary
ASTM D5430-26 [42]	Visual fabric inspection/grading context
GS1 Global Traceability Standard [14]	Product/batch/instance traceability principles
PostgreSQL transactions/locking [43]	Reservation/dispatch concurrency
PostgreSQL constraints [27]	DB-enforced invariants
SQLite transactions/WAL [25]	Local fallback concurrency
SQLite foreign keys [44]	Local relational integrity
RFC 9110 [22]	HTTP idempotency semantics
OWASP validation guidance [45]	Server-side validation
OWASP query parameterization [46]	SQL-injection prevention
OWASP authorization guidance [47]	Role/action enforcement

The project should be judged internally by one brutal question before you call it complete:

> **Can a judge create one buyer order, watch it consume/reserve real inventory, move through real production state, intentionally fail quality, see the ERP prevent an illegal dispatch, fix the problem, dispatch the correct rolls, and trace the final physical roll back through the factory—without us explaining away any broken step?**

If the answer is yes, stop adding features. That product is much stronger than a half-built "complete textile ERP."

---

[1] [35] Odoo x NMIT Bangalore Hackathon 26
https://hackathon.odoo.com/event/odoo-x-nmit-bangalore-hackathon-2026-38/register

[2] [34] Odoo Hackathon 2026
https://hackathon.odoo.com/

[3] [6] [36] Bill of materials — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/manufacturing/basic_setup/bill_configuration.html?utm_source=chatgpt.com

[4] Learning Hub | CottonWorks™ CottonWorks™ – Cotton Textiles Learning Hub for Professionals
https://cottonworks.com/learning-hub/?utm_source=chatgpt.com

[5] [40] Global Standards - GOTS – How the system works | Global Standards
https://global-standards.org/our-standards/gots/how-it-works?utm_source=chatgpt.com

7 Vendor bills — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/finance/accounting/vendor_bills.html?utm_source=chatgpt.com

8 Three-step receipt — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/inventory/management/shipments_deliveries/receipts_three_steps.html?utm_source=chatgpt.com

9 20 At confirmation reservation — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/inventory/shipping_receiving/reservation_methods/at_confirmation.html?utm_source=chatgpt.com

10 Regras de reposição — documentação Odoo 19.0
https://www.odoo.com/documentation/19.0/pt_BR/applications/inventory_and_mrp/inventory/warehouses_storage/replenishment/reordering_rules.html?utm_source=chatgpt.com

11 33 Work centers — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/manufacturing/advanced_configuration/using_work_centers.html?utm_source=chatgpt.com

12 21 Quality checks — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/quality/quality_management/quality_checks.html?utm_source=chatgpt.com

13 41 ISO 8498:1990 - Woven fabrics — Description of defects — Vocabulary
https://www.iso.org/standard/15708.html?utm_source=chatgpt.com

14 GS1 Global Traceability Standard | GS1
https://www.gs1.org/standards/gs1-global-traceability-standard/current-standard?utm_source=chatgpt.com

15 Manufacture with lots and serial numbers — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/manufacturing/workflows/manufacture_lots_serials.html?utm_source=chatgpt.com

16 Process repair orders — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/repairs/repair_orders.html?utm_source=chatgpt.com

17 27 PostgreSQL: Documentation: 18: 5.5. Constraints
https://www.postgresql.org/docs/current/ddl-constraints.html?utm_source=chatgpt.com

18 44 SQLite Foreign Key Support
https://www.sqlite.org/foreignkeys.html?utm_source=chatgpt.com

19 Failure locations — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/quality/quality_management/failure_locations.html?utm_source=chatgpt.com

22 RFC 9110: HTTP Semantics
https://www.rfc-editor.org/rfc/rfc9110.html?utm_source=chatgpt.com

23 PostgreSQL: Documentation: 17: 13.3. Explicit Locking
https://www.postgresql.org/docs/17/explicit-locking.html?utm_source=chatgpt.com

24 PostgreSQL: Documentation: 16: 13.2. Transaction Isolation
https://www.postgresql.org/docs/16/transaction-iso.html?utm_source=chatgpt.com

25 SQLite Query Language: BEGIN TRANSACTION
https://system.data.sqlite.org/home/doc/4deb290d232e5850/Doc/Extra/Core/lang_transaction.html?utm_source=chatgpt.com

26 SQLite Query Language: upsert
https://www2.sqlite.org/lang_UPSERT.html?utm_source=chatgpt.com

28 Authorization Patterns - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Patterns_Cheat_Sheet.html?utm_source=chatgpt.com

29 45 Input Validation - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html?utm_source=chatgpt.com

30 46 Query Parameterization - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet?utm_source=chatgpt.com

31 Password Storage - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html?utm_source=chatgpt.com

32 PostgreSQL: Documentation: 9.0: Constraints
https://www.postgresql.org/docs/9.0/ddl-constraints.html?utm_source=chatgpt.com

37 Manufacturing orders and work orders — Odoo master documentation
https://www.odoo.com/documentation/master/applications/inventory_and_mrp/manufacturing/basic_setup/manufacturing_work_orders.html?utm_source=chatgpt.com

38 Replenishment — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/inventory/warehouses_storage/replenishment.html?utm_source=chatgpt.com

39 Scrap during manufacturing — Odoo 19.0 documentation
https://www.odoo.com/documentation/19.0/applications/inventory_and_mrp/manufacturing/workflows/scrap_manufacturing.html?utm_source=chatgpt.com

42 D5430 Standard Test Methods for Visually Inspecting and Grading Fabrics
https://store.astm.org/standards/d5430?utm_source=chatgpt.com

43 PostgreSQL: Documentation: 18: 13.2. Transaction Isolation
https://www.postgresql.org/docs/current/transaction-iso.html?utm_source=chatgpt.com

47 Authorization - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html?utm_source=chatgpt.com